UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

GUÍA DE LABORATORIO N.º 3

Automatización y Calidad de Código

I. Propósito de la sesión

Al finalizar el laboratorio, el estudiante habrá construido una mini-herramienta de automatización de archivos con interfaz de línea de comandos (pathlib + shutil + argparse), cubierta por pruebas unitarias con pytest usando el fixture tmp_path, y documentada con docstrings en estilo Google y type hints — cerrando así el entregable completo de la Unidad I.

II. Requisitos previos

- Repositorio tecemer-lab1 de las Semanas 1 y 2, con su entorno virtual (.venv) y el remoto de GitHub configurados.
- Haber completado las Guías de Laboratorio N.º 1 y N.º 2.

III. Estructura de la sesión (4 horas)

Parte	Tema	Nivel	Duración
1	pathlib + shutil: organizador de archivos	Básico	60 min
2	Pruebas unitarias con pytest y tmp_path	Intermedio	90 min
3	Documentación técnica y consolidación (cierre de Unidad I)	Avanzado	90 min
Total			**240 min**

IV. Desarrollo paso a paso

PARTE 1 — PATHLIB + SHUTIL: ORGANIZADOR DE ARCHIVOS

Paso 1.1 — Preparar una carpeta de prueba

Ubíquese en tecemer-lab1 y cree una carpeta con archivos de muestra para probar el organizador manualmente durante el desarrollo:

```
cd tecemer-lab1
mkdir carpeta_prueba
ni carpeta_prueba\foto.jpg -ItemType File        # macOS/Linux: touch carpeta_prueba/foto.jpg
```

```
ni carpeta_prueba\informe.pdf -ItemType File        # macOS/Linux: touch carpeta_prueba/informe.pdf
ni carpeta_prueba\video.mp4 -ItemType File          # macOS/Linux: touch carpeta_prueba/video.mp4
```

Paso 1.2 — Núcleo de clasificación con pathlib

Cree el archivo organizador.py con el siguiente contenido (este es el archivo base; todos los pasos siguientes de la Parte 1 se AGREGAN a este mismo archivo):

```
from pathlib import Path

CATEGORIAS = {
    ".pdf": "Documentos", ".docx": "Documentos", ".txt": "Documentos",
    ".jpg": "Imagenes", ".jpeg": "Imagenes", ".png": "Imagenes",
    ".mp4": "Videos", ".mov": "Videos",
    ".zip": "Comprimidos", ".rar": "Comprimidos",
}


def clasificar(archivo):
    return CATEGORIAS.get(archivo.suffix.lower(), "Otros")
```

Paso 1.3 — Mover archivos con shutil

Agregue al FINAL de organizador.py la función que organiza toda la carpeta:

```
import shutil


def organizar_carpeta(carpeta, simulacion=False):
    movidos = 0
    for archivo in carpeta.iterdir():
        if archivo.is_dir() or archivo.name.startswith("."):
            continue
        categoria = clasificar(archivo)
        destino_carpeta = carpeta / categoria
        destino = destino_carpeta / archivo.name

        if simulacion:
            print(f"[SIMULACION] {archivo.name} -> {categoria}")
        else:
            destino_carpeta.mkdir(exist_ok=True)
            shutil.move(str(archivo), str(destino))
            print(f"Movido: {archivo.name} -> {categoria}")
        movidos += 1
    return movidos
```

Nota: *El import shutil se agrega al inicio del bloque nuevo por claridad, pero Python solo exige que los import existan en algún punto antes de su primer uso; puede subirlo junto a from pathlib import Path si lo prefiere.*

Paso 1.4 — Interfaz de línea de comandos con argparse (20 min)

Agregue al FINAL de organizador.py el punto de entrada con argparse, incluyendo el modo --dry-run:

```
import argparse


def main():
    parser = argparse.ArgumentParser(description="Organiza archivos por tipo.")
    parser.add_argument("carpeta", type=Path, help="Carpeta a organizar")
    parser.add_argument("--dry-run", action="store_true", help="Solo simula, no mueve archivos")
    args = parser.parse_args()

    total = organizar_carpeta(args.carpeta, simulacion=args.dry_run)
    print(f"Total de archivos procesados: {total}")


if __name__ == "__main__":
    main()
```

Pruebe el script manualmente en modo simulación y luego en modo real:

```
.venv\Scripts\python organizador.py carpeta_prueba --dry-run
.venv\Scripts\python organizador.py carpeta_prueba
# macOS/Linux: .venv/bin/python organizador.py carpeta_prueba --dry-run
```

Verificación: *Checkpoint de Parte 1: en modo --dry-run, los archivos NO deben moverse (solo se imprime lo que haría). Sin esa bandera, deben aparecer las subcarpetas Imagenes, Documentos y Videos dentro de carpeta_prueba.*

PARTE 2 — PRUEBAS UNITARIAS CON PYTEST Y TMP_PATH

Paso 2.1 — Instalar pytest

```
.venv\Scripts\pip install pytest
.venv\Scripts\pip freeze > requirements.txt
# macOS/Linux: .venv/bin/pip install pytest
```

Paso 2.2 — Primera prueba: clasificación y movimiento

Cree el archivo test_organizador.py (nuevo archivo, distinto de organizador.py) con el siguiente contenido:

```
from organizador import clasificar, organizar_carpeta


def test_clasificar_reconoce_imagen(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")
```

```
    assert clasificar(archivo) == "Imagenes"


def test_organizar_mueve_archivo_a_subcarpeta(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path)

    assert (tmp_path / "Imagenes" / "foto.jpg").exists()
    assert not archivo.exists()
```

Paso 2.3 — Segunda prueba: modo simulación no debe mover nada

Agregue al FINAL de test_organizador.py:

```
def test_organizar_en_modo_simulacion_no_mueve_nada(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path, simulacion=True)

    assert archivo.exists()
    assert not (tmp_path / "Imagenes").exists()
```

Paso 2.4 — Ejecutar pytest y leer el reporte

```
.venv\Scripts\pytest -v
# macOS/Linux: .venv/bin/pytest -v
```

Verificación: *Salida esperada: 3 pruebas recolectadas (collected 3 items), las tres con estado PASSED, y un resumen final 3 passed en unos pocos segundos.*

Paso 2.5 — Checkpoint de Parte 2

Confirme los avances en Git antes de continuar:

```
git add organizador.py test_organizador.py requirements.txt
git commit -m "feat: agrega organizador de archivos con pruebas pytest"
```

PARTE 3 — DOCUMENTACIÓN TÉCNICA Y CONSOLIDACIÓN

Paso 3.1 — Instalar docformatter

```
.venv\Scripts\pip install docformatter
# macOS/Linux: .venv/bin/pip install docformatter
```

Paso 3.2 — Reescribir organizador.py con docstrings, estilo Google y type hints

Reemplace TODO el contenido de organizador.py por esta versión final documentada (mismo comportamiento, con documentación agregada):

```
"""Organiza los archivos de una carpeta en subcarpetas segun su extension."""
import argparse
import shutil
from pathlib import Path

CATEGORIAS = {
    ".pdf": "Documentos", ".docx": "Documentos", ".txt": "Documentos",
    ".jpg": "Imagenes", ".jpeg": "Imagenes", ".png": "Imagenes",
    ".mp4": "Videos", ".mov": "Videos",
    ".zip": "Comprimidos", ".rar": "Comprimidos",
}


def clasificar(archivo: Path) -> str:
    """Determina la categoria de un archivo segun su extension.

    Args:
        archivo: Ruta del archivo a clasificar.

    Returns:
        Nombre de la categoria correspondiente, u "Otros" si la
        extension no esta registrada.
    """
    return CATEGORIAS.get(archivo.suffix.lower(), "Otros")


def organizar_carpeta(carpeta: Path, simulacion: bool = False) -> int:
    """Organiza los archivos de una carpeta en subcarpetas por categoria.

    Args:
        carpeta: Carpeta cuyos archivos se van a organizar.
        simulacion: Si es True, solo muestra que haria sin mover archivos.

    Returns:
        Numero de archivos movidos (o que se moverian, en modo
        simulacion).
    """
    movidos = 0
    for archivo in carpeta.iterdir():
        if archivo.is_dir() or archivo.name.startswith("."):
            continue
        categoria = clasificar(archivo)
        destino_carpeta = carpeta / categoria
        destino = destino_carpeta / archivo.name

        if simulacion:
            print(f"[SIMULACION] {archivo.name} -> {categoria}")
        else:
            destino_carpeta.mkdir(exist_ok=True)
            shutil.move(str(archivo), str(destino))
            print(f"Movido: {archivo.name} -> {categoria}")
        movidos += 1
    return movidos
```

```
def main() -> None:
    """Punto de entrada de linea de comandos del organizador."""
    parser = argparse.ArgumentParser(description="Organiza archivos por tipo.")
    parser.add_argument("carpeta", type=Path, help="Carpeta a organizar")
    parser.add_argument(
        "--dry-run", action="store_true", help="Solo simula, no mueve archivos"
    )
    args = parser.parse_args()

    total = organizar_carpeta(args.carpeta, simulacion=args.dry_run)
    print(f"Total de archivos procesados: {total}")


if __name__ == "__main__":
    main()
```

Verificación: *Vuelva a ejecutar pytest -v para confirmar que las 3 pruebas de la Parte 2 siguen pasando: agregar documentación no debe cambiar el comportamiento del código.*

Paso 3.3 — Formatear los docstrings con docformatter

```
.venv\Scripts\docformatter --in-place organizador.py
# macOS/Linux: .venv/bin/docformatter --in-place organizador.py
```

Paso 3.4 — README de cierre de la Unidad I

Agregue al README.md una sección final que documente el entregable completo de la unidad:

```
## Cierre de la Unidad I — Semana 3

Herramienta de automatización: organizador.py clasifica y mueve archivos
de una carpeta en subcarpetas por tipo (Documentos, Imagenes, Videos,
Comprimidos, Otros), con modo de simulacion (--dry-run) mediante argparse.

Uso:
```
python organizador.py <carpeta> [--dry-run]
```

Pruebas: test_organizador.py cubre clasificacion, movimiento real y modo
simulacion, usando la fixture tmp_path de pytest para no afectar el
sistema de archivos real. Ejecutar con: pytest -v
```

Paso 3.5 — Commit final y cierre de la Unidad I

```
git add organizador.py README.md
git commit -m "docs: documenta organizador con estilo Google y cierra la Unidad I"
git push origin main
```

Copie el enlace de su repositorio (con el historial acumulado de las 3 semanas) y súbalo al aula virtual como evidencia de entrega de la Unidad I completa.

V. Entregable de la sesión

El mismo repositorio de GitHub de las Semanas 1 y 2, con organizador.py (documentado en estilo Google, con type hints y formateado con docformatter) y test_organizador.py (mínimo 3 pruebas con tmp_path, todas en verde), el README.md con la sección de cierre de la Unidad I, y un commit final en formato Conventional Commits.